Daily Quant Digest: 2026-10-29

Foundations Restart, day 4. Yesterday we counted queues, saw that a price is really two prices, and rearranged an array in place. Today: counting when order does **not** matter, how currency prices are written and read, and the hash set, the most useful container in interviews.

Today's topics

Track	Topic	Difficulty
Probability & Statistics	FR Lesson 4: counting II: choosing (combinations), $\binom{n}{k}$, Pascal's triangle, cards and committees	Easy
Derivatives	FR Lesson 4: FX basics: currency pairs, base and quote currency, pips, how EURUSD and USDJPY are quoted	Easy
Algorithms & DS	FR Lesson 4: hash maps and sets; longest consecutive sequence	Easy-Medium
Music	Jimi Hendrix, "All Along the Watchtower" (1968)	

---

1. Probability & Statistics: Choosing, When Order Does Not Matter

> **Where we are.** Lesson 1: with equally likely outcomes, $P(A) = |A|/|\Omega|$ (people in the group divided by people in the room). Lesson 2: "or", "and", "not" and inclusion-exclusion. Lesson 3: the multiplication principle, $n!$ ways to arrange n things in a row, and $P(n,k) = n!/(n-k)!$ ways to fill k chairs in order. At the end of Lesson 3 we noticed that a podium (gold, silver, bronze) is not the same as a "top-3 team". Today we count teams.

1.1 The central picture: the queue and the bag

Imagine 10 traders on a desk. The boss walks along and taps three of them on the shoulder, one after another: "you, you and you, you are on the new FX committee."

If we write down the taps **in order**, we get a **queue**: first tap, second tap, third tap. Lesson 3 counts these: $10 \times 9 \times 8 = 720$ queues.

But the committee does not care who was tapped first. The three names go into a **bag**, and a bag has no order. Shake it and it is still the same bag. So many different queues give the same bag. How many?

Take the bag {Amy, Ben, Cat}. Every way of lining those three up is a queue that ends in this bag: ABC, ACB, BAC, BCA, CAB, CBA. That is $3! = 6$ queues. And this is true for **every** bag of three: each bag is hit by exactly 6 queues. So

$$\text{number of bags} = \frac{720}{6} = 120.$$

That one division is the whole lesson. Everything else is practice.

In the picture	In the maths
The 10 traders	The n items we choose from
Three taps on the shoulder	Choosing k items
The queue (tap order matters)	An arrangement, counted by $P(n, k)$
The bag (only who is in it)	A **combination** (a subset)
Shaking the bag	Rearranging the k chosen items: $k!$ ways
"Each bag is hit by $k!$ queues"	Divide by $k!$

1.2 The formula, one step at a time

In words: count the ordered picks, then divide by the number of orders each bag can be lined up in.

$$\binom{n}{k} = \frac{P(n, k)}{k!} = \frac{n!}{k!\,(n-k)!}.$$

Symbols. n = how many items there are. k = how many we choose. $\binom{n}{k}$ = the number of different groups of size k, with no order.

Read aloud: "n choose k."

Why the second form? From Lesson 3, $P(n, k) = n!/(n-k)!$. Dividing that by $k!$ just puts $k!$ next to $(n-k)!$ in the bottom.

With numbers. $\binom{10}{3} = \dfrac{10!}{3!\,7!}$. Do not compute $10!$; cancel the $7!$ first: $\dfrac{10 \times 9 \times 8}{3 \times 2 \times 1} = \dfrac{720}{6} = 120$. The quick rule: k **numbers counting down from** n **on top,** $k!$ **on the bottom.**

A small check you can do by hand. Choose 2 of the 4 letters A, B, C, D. The formula says $\binom{4}{2} = \frac{4 \times 3}{2} = 6$. List them: AB, AC, AD, BC, BD, CD. Six. Good.

1.3 Two facts that make sense in words

Fact 1: choosing who goes in is the same as choosing who stays out.

$$\binom{n}{k} = \binom{n}{n-k}.$$

Picking 3 of the 10 traders for the committee is the same act as picking the 7 who are **not** on it. Every committee gives one "left-out" group, and back again. So $\binom{10}{3} = \binom{10}{7} = 120$. Useful trick: $\binom{50}{48}$ looks scary, but it is $\binom{50}{2} = \frac{50 \times 49}{2} = 1{,}225$.

Fact 2: Pascal's rule (is Amy in, or not?).

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}.$$

Point at one trader, Amy. Every committee either **contains Amy** or **does not** (Lesson 2: two groups with no overlap, so we add).

- Amy is in: we still need $k-1$ more people from the other $n-1$. That is $\binom{n-1}{k-1}$.
- Amy is out: all k seats go to the other $n-1$. That is $\binom{n-1}{k}$.

With numbers: $\binom{10}{3} = \binom{9}{2} + \binom{9}{3} = 36 + 84 = 120$. It matches.

1.4 Pascal's triangle

Pascal's rule says each number is the sum of the two above it. Write row n as $\binom{n}{0}, \binom{n}{1}, \ldots, \binom{n}{n}$:

n	**row**
0	1
1	1 1
2	1 2 1
3	1 3 3 1
4	1 4 6 4 1
5	1 5 10 10 5 1
6	1 6 15 20 15 6 1

Each row reads the same forwards and backwards: that is Fact 1. The ends are always 1: one way to choose nobody, one way to choose everybody. And each row adds up to a power of 2. Row 6 adds to $64 = 2^6$. Why? Every trader is either in or out of a group: 2 choices each, 2^6 groups of any size in total (Lesson 3).

1.5 Cards: the most common use in interviews

A 5-card poker hand is a **bag**: the order you were dealt the cards does not matter. So the room is

$$|\Omega| = \binom{52}{5} = \frac{52 \times 51 \times 50 \times 49 \times 48}{5 \times 4 \times 3 \times 2 \times 1} = 2{,}598{,}960 \text{ hands, all equally likely.}$$

The method for the group is always the same: **build the hand in independent stages, choose inside each stage, multiply** (Lesson 3).

Exactly two aces. Stage 1: which 2 of the 4 aces, $\binom{4}{2} = 6$. Stage 2: the other 3 cards from the 48 non-aces, $\binom{48}{3} = 17{,}296$. Group size $6 \times 17{,}296 = 103{,}776$, so $P = 103{,}776 / 2{,}598{,}960 \approx 0.040$, about 1 hand in 25.

1.6 Common mistakes

- **Mixing the queue and the bag.** Ask one question: "if I swap two of the chosen items, is it a different outcome?" Yes: arrangements ($P(n,k)$). No: combinations ($\binom{n}{k}$). And use the **same** kind for the room and for the group, or the division is meaningless.
- **"At least one" by stages.** "At least one ace: choose 1 ace (4 ways), then any 4 of the other 51." This counts a hand with two aces **twice** (once for each ace you could have called "the first"). Count the complement instead (Lesson 2): hands with no ace are $\binom{48}{5} = 1{,}712{,}304$, so $P(\text{at least one ace}) = 1 - 1{,}712{,}304 / 2{,}598{,}960 \approx 0.341$.
- **Where the picture stops working.** "Divide by $k!$" works because every bag is hit by **the same** number of queues. If items repeat (choosing letters from MISSISSIPPI), bags are hit unevenly, and you need the tools of Lesson 5.

1.7 Self-check

1. How many ways can you pick 6 numbers from 1 to 49 in a lottery (order does not matter)?
2. A desk has 6 men and 5 women. A committee of 4 is chosen at random. What is the chance it has exactly 2 women?
3. Without a calculator: what is $\binom{20}{19}$?

1.8 Interview question: three poker hands

(Card counting of this kind is a standard warm-up at SIG, Optiver and Jane Street. They want to see you split a hand into stages cleanly.)

You are dealt 5 cards from a well-shuffled 52-card deck.

a. What is the probability that all 5 cards are the same suit (a "flush", counting straight flushes too)?

b. What is the probability of a full house (three cards of one rank and two of another, like three kings and two sevens)?

c. Which is more likely, and roughly how many times more?

1.9 Solution

The room is $\binom{52}{5} = 2{,}598{,}960$ bags.

(a) Flush, in two stages.

- Stage 1: pick the suit: 4 ways.
- Stage 2: pick 5 of that suit's 13 cards: $\binom{13}{5} = \frac{13\times12\times11\times10\times9}{120} = 1{,}287$.

Group: $4 \times 1{,}287 = 5{,}148$. So $P = 5{,}148/2{,}598{,}960 \approx 0.00198$, about 1 in 505.

(b) Full house, in four stages. Order the stages carefully, because "the rank for three" and "the rank for two" play **different roles**.

- Stage 1: the rank of the three: 13 ways.
- Stage 2: which 3 of its 4 suits: $\binom{4}{3} = 4$.
- Stage 3: the rank of the pair, any of the other 12 ranks: 12 ways.
- Stage 4: which 2 of its 4 suits: $\binom{4}{2} = 6$.

Group: $13 \times 4 \times 12 \times 6 = 3{,}744$. So $P = 3{,}744/2{,}598{,}960 \approx 0.00144$, about 1 in 694.

The trap. Many people write $\binom{13}{2}$ for "choose the two ranks". That treats {kings, sevens} as a bag, so "three kings and two sevens" and "three sevens and two kings" are counted as one. They are different hands. The ranks are a **queue** here (first role, second role): 13×12, not $\binom{13}{2} = 78$. The wrong answer is exactly half the right one.

(c) The flush is more likely: $5{,}148/3{,}744 = 1.375$, about 1.4 times. This is why a full house beats a flush in poker: rarer hands win.

Interview trick. For (a) skip the counting: the first card can be anything; each later card must match its suit. $1 \times \frac{12}{51} \times \frac{11}{50} \times \frac{10}{49} \times \frac{9}{48} \approx 0.00198$. Same answer, and it is the "queue" view of the same bag: Lesson 3's chairs, filled one at a time.

Sanity check. Two different methods agree for (a). For (b), each full house is described by exactly one recipe (rank of the three, its suits, rank of the pair, its suits), so nothing is counted twice or missed. And I listed all 2,598,960 hands by computer: 5,148 flushes, 3,744 full houses, and 103,776 hands with exactly two aces, matching every count above.

Self-check answers. (1) $\binom{49}{6} = 13{,}983{,}816$. (2) Stage 1: 2 of 5 women, $\binom{5}{2} = 10$. Stage 2: 2 of 6 men, $\binom{6}{2} = 15$. Group 150. Room $\binom{11}{4} = 330$. $P = 150/330 = 5/11 \approx 0.45$. (3) $\binom{20}{19} = \binom{20}{1} = 20$ (choose the one who stays out).

---

2. Derivatives: How Currency Prices Are Written and Read

> **Where we are.** Lesson 1: money grows when lent. Lesson 2: future money is worth less today, using discount factors. Lesson 3: every price is two prices, a bid and an ask, set by a market maker; being long means you gain when the price rises, being short means you gain when it falls. Today we apply all of that to the biggest market in the world: foreign exchange (FX).

2.1 The scene

It is 9 a.m. on an FX desk in Hong Kong. Three calls come in.

- A Hong Kong importer of German machines needs to **buy EUR 5 million** to pay its supplier. It asks for a price in **EURUSD**.
- A Japanese life insurer has received dollars from bonds that matured and wants to **sell USD 10 million** for yen. It asks for **USDJPY**.
- A hedge fund thinks the yen will get stronger against the euro. It asks for **EURJPY**.

Three clients, three different-looking prices. Today's goal: read each one without thinking twice.

2.2 The central picture: a price tag on a shelf

Every FX price is a **price tag**. "EURUSD 1.1650" means:

$$1 \text{ EUR} = 1.1650 \text{ USD}.$$

The first currency (EUR) is the **thing on the shelf**: the item being priced. It is called the **base currency**. The second currency (USD) is the **money in your wallet**: what you pay with. It is called the **quote currency** (or "terms" currency). A price tag on an apple says "1 apple = 0.80 dollars". An FX price tag says "1 euro = 1.1650 dollars". Same idea.

In the picture	In the FX market
The item on the shelf	The **base** currency (first in the name)
The money in your wallet	The **quote** currency (second)
The price on the tag	How many quote units buy 1 base unit
Buying the item	Buying the base currency (going long the pair)
A cent on the price tag	A **pip**: the smallest usual price step

The key consequence. When the number goes **up**, the item on the shelf got more expensive: the **base** currency got **stronger**. EURUSD from 1.1650 to 1.1700: the euro gained, the dollar lost. USDJPY from 150 to 160: the **dollar** gained and the **yen** lost, because the yen is the wallet money. This is the most common confusion among beginners: "USDJPY up" means "yen weaker".

Who is on the shelf? Market convention. You never see "USDEUR" quoted. Dealers agree on an order, roughly: EUR, GBP, AUD, NZD, USD, CAD, CHF, JPY. The currency that comes first in this list is the base. So it is EURUSD, GBPUSD, USDJPY, USDCAD, EURJPY. Hong Kong's own pair is USDHKD, which the Hong Kong Monetary Authority keeps between 7.75 and 7.85 (the peg).

2.3 Pips: the cents of FX

In words: a pip is the step dealers count in. For most pairs it is the **4th decimal place**, 0.0001. For pairs with JPY as the quote currency, it is the **2nd decimal place**, 0.01, because one yen is a small amount of money.

EURUSD 1.1650 to 1.1652 is **2 pips**. USDJPY 152.30 to 152.32 is also **2 pips**.

Dealers also save breath. In "1.1650 / 1.1652", the "1.16" part (the **big figure**) is known to everyone, so the dealer just says "**fifty, fifty-two**".

How much money is one pip? The formula, in words: **one pip of price, times the amount of base currency you hold, is the money you gain or lose, paid in the quote currency.**

$$\text{pip value (in quote currency)} = \text{pip size} \times \text{notional in base currency}.$$

Symbols: pip size = 0.0001 (or 0.01 for JPY pairs). Notional = how much of the base currency the trade is for.

With numbers.

- EUR 5 million in EURUSD: $0.0001 \times 5{,}000{,}000 = 500$, so **USD 500 per pip**. EUR 10 million is USD 1,000 per pip, which is why dealers like round numbers.
- USD 10 million in USDJPY: $0.01 \times 10{,}000{,}000 = 100{,}000$, so **JPY 100,000 per pip**. In dollars, at 152.30, that is $100{,}000 / 152.30 \approx$ USD 656.60.

Notice that the pip value is naturally in the **quote** currency (the wallet money). To get it in any other currency, convert at today's rate.

2.4 Flipping a price tag

Sometimes you need the price "the other way round". If 1 EUR = 1.1650 USD, then 1 USD = $1 / 1.1650 \approx 0.8584$ EUR. Easy.

But remember Lesson 3: the dealer shows **two** prices. "EURUSD 1.1650 / 1.1652" means the dealer **buys** EUR at 1.1650 (bid) and **sells** EUR at 1.1652 (ask). Buying EUR is the same act as selling USD. So when we flip the tag:

- The dealer's bid for USD (in EUR) comes from the dealer's **ask** for EUR: $1 / 1.1652 \approx 0.85822$.
- The dealer's ask for USD comes from the **bid** for EUR: $1 / 1.1650 \approx 0.85837$.

The sides **swap** when you flip. The dealer still wins the spread either way, which is a good check: the bid must always be the lower number.

2.5 Cross rates: going through the dollar

The hedge fund wants EURJPY. Most FX trading goes through the dollar, so a dealer builds EURJPY from two dollar pairs. The trick is to see the units cancel, like a chain of exchange booths:

$$\frac{\text{USD}}{\text{EUR}} \times \frac{\text{JPY}}{\text{USD}} = \frac{\text{JPY}}{\text{EUR}}, \qquad \text{EURJPY} = \text{EURUSD} \times \text{USDJPY}.$$

Say EURUSD is 1.1650 / 1.1652 and USDJPY is 152.30 / 152.32.

- **Dealer's bid for EUR in JPY.** The client sells EUR, gets USD at the EURUSD **bid** (1.1650), then sells those USD for JPY at the USDJPY **bid** (152.30): $1.1650 \times 152.30 = 177.43$.
- **Dealer's ask.** The client buys EUR: pays the ask at each step: $1.1652 \times 152.32 \approx 177.48$.

So EURJPY is about **177.43 / 177.48**, a spread of about 5 pips, wider than either leg (2 pips each). You pay two spreads because you cross two booths. That is why direct crosses trade with wider spreads than the two dollar pairs.

2.6 A worked trade

Trade 1, EURUSD. The importer buys EUR 5 million at the dealer's ask, **1.1652**. Suppose the importer had instead been a speculator, and a week later EURUSD is 1.1700 / 1.1702. Closing means **selling** EUR, at the bid, 1.1700.

- Move: $1.1700 - 1.1652 = 0.0048$, which is **48 pips**.
- P&L: $48 \times 500 = 24{,}000$, so **USD 24,000**. Check directly: $0.0048 \times 5{,}000{,}000 = 24{,}000$.

Trade 2, USDJPY. The insurer sells USD 10 million at the bid, **152.30**: it is now short dollars, long yen. Later USDJPY is 150.80 / 150.82. Buying back the dollars costs the ask, 150.82.

- Move: $152.30 - 150.82 = 1.48$ yen per dollar, which is **148 pips**.
- P&L: $148 \times 100{,}000 = 14{,}800{,}000$, so **JPY 14.8 million**, about USD 98,130 at 150.82.

When it goes wrong: the yen carry unwind, 2024. For years, investors borrowed yen at near-zero rates, sold them, and bought higher-yielding currencies: short JPY, long the rest. USDJPY climbed to around 162 in early July 2024. Then the Bank of Japan raised rates and US data softened, and everyone tried to buy yen back at once. By 5 August 2024 USDJPY was near 142, a fall of more than 12% in about a month, and Japan's Nikkei index

fell over 12% that single day. A short-yen position of USD 10 million would have lost roughly 2,000 pips, about JPY 200 million. Pip values are small; moves of 2,000 pips are not.

Where the picture stops working. A price tag on an apple measures the apple in money. In FX both sides are money, so there is no fixed "price of the dollar". A pair only tells you the strength of one currency **relative** to another. The dollar can be up against the yen and down against the euro on the same day.

2.7 Likely interview questions

1. **"You are long EUR 2 million in EURUSD from 1.0850. It is now 1.0920. What is your P&L?"** 70 pips, USD 200 per pip, so **+USD 14,000** (paid in USD, the quote currency).
2. **"USDJPY goes from 150 to 160. By how much did the dollar strengthen, and by how much did the yen weaken?"** The dollar buys $160/150 - 1 \approx +6.67\%$ more yen. The yen is worth $\frac{1/160}{1/150} - 1 = -6.25\%$ in dollars. Not the same number: percentages on a flipped price tag are not symmetric.
3. **"EURUSD is 1.1650/52 and USDJPY is 152.30/32. Make me a price in EURJPY."** Bid $= 1.1650 \times 152.30 = 177.43$, ask $= 1.1652 \times 152.32 \approx 177.48$. Bid times bid, ask times ask, because both legs move in the same direction for the client.

---

3. Algorithms & Data Structures: Hash Sets and the Longest Consecutive Run

> **Where we are.** Lesson 1: Big-O, and why $n \log n$ beats n^2. Lesson 2: `std::vector`, `std::string`, references and loops. Lesson 3: scanning an array and updating it in place. So far, to ask "is the number 42 in my array?" we had to scan the whole array: $O(n)$ per question. Today we meet a container that answers that question in about **one step**.

3.1 The central picture: the coat check

At a big hotel party, 1,000 guests hand in their coats. A bad cloakroom piles the coats on one long rail in arrival order. To find your coat, the attendant walks the rail: up to 1,000 checks.

A good cloakroom has 1,000 numbered hooks. When you hand in your coat, the attendant looks at your name, does a quick calculation that turns it into a hook number, and hangs the coat there. Later, the same calculation sends him straight back to the same hook. One step, however many coats there are.

That calculation is a **hash function**, and the cloakroom is a **hash table**.

In the picture	In C++
The coat (or the guest's name)	A **key**, e.g. an `int`
The hook-number calculation	The **hash function**
The hooks	The **buckets** inside the table
"Is Mr. Lee's coat here?"	`s.count(key)` : average $O(1)$
Two guests sent to the same hook	A **collision**: a few coats share a hook
Ticket + coat (name and coat)	`unordered_map` : key with a value
Just the name on a guest list	`unordered_set` : key only

Collisions. Sometimes two names give the same hook. Then a few coats hang on that hook and the attendant checks them one by one. If the calculation spreads names evenly, each hook has very few coats, so a lookup is **on average** $O(1)$. In the worst case (everyone on one hook) it is $O(n)$, but a good hash and a table that grows as it fills make that very rare. In C++ the table grows automatically.

Where the picture stops working. A cloakroom keeps hooks in number order, but a hash table gives you **no useful order** at all. Looping over an `unordered_set` visits keys in a scrambled order. If you need sorted order, use `std::set` (Lesson 17 of this track).

The two containers.

- `std::unordered_set<int>`: a guest list. Insert, remove, and "is this in?" are average $O(1)$. Duplicates are not stored twice.
- `std::unordered_map<std::string, double>`: a coat check, a key with a value. `m["EURUSD"] = 1.1650;` stores, `m["EURUSD"]` reads.

3.2 The problem

Longest consecutive sequence. You are given an unsorted array of integers `nums`. Return the length of the longest run of **consecutive integers** (like 7, 8, 9, 10) whose members all appear in the array. The order in the array does not matter, and duplicates count once. Your algorithm should run in $O(n)$ time.

Trading picture: `nums` are the day numbers on which a strategy made money, in random order. How long was its longest winning streak?

Constraints. $0 \le n \le 10^5$, and $-10^9 \le$ `nums[i]` $\le 10^9$.

Examples.

`nums`	Answer	Why
`[100, 4, 200, 1, 3, 2]`	4	1, 2, 3, 4
`[0, 3, 7, 2, 5, 8, 4, 6, 0, 1]`	9	0 to 8 (the second 0 counts once)
`[]`	0	no numbers
`[1, 2, 0, 1]`	3	0, 1, 2

3.3 Brute force first

Idea 1: try every start, check with a scan. For each number x, see if $x + 1$ is in the array (scan, $O(n)$), then $x + 2$, and so on. A run can be up to n long, and we start from all n numbers: up to $n \times n \times n = O(n^3)$. With $n = 10^5$ that is 10^{15} steps: hopeless.

Idea 2: sort first. After sorting, consecutive numbers sit next to each other. Remove duplicates and walk once, extending the current run when `nums[i] == nums[i-1] + 1` and restarting otherwise. That is $O(n \log n)$ for the sort plus $O(n)$ for the walk. A fine answer, and we keep it in the code as a checker. But the problem asks for $O(n)$, and sorting is the only slow part. What did we really need from sorting? Only one question: "**is $x + 1$ present?**" That is exactly what a hash set answers in $O(1)$.

3.4 The insight: only start counting at the front of a queue

Step 1. Put every number in an `unordered_set`. Now "is x present?" costs $O(1)$.

Step 2 (first try). For every x, walk right: $x + 1$? $x + 2$? ... Each step is $O(1)$ now. But this is still slow. In `[1, 2, 3, ..., 100000]`, we walk 100,000 steps from 1, then 99,999 from 2, then 99,998 from 3... About $n^2/2$ steps in total. We keep re-walking the same run from the middle.

Step 3 (the insight). Think of each run as a **queue of people standing in a line**. To measure a queue, you walk to the person at the **front** and count backwards to the end. You never start measuring from someone in the middle.

How do you know x is at the front? **Its left neighbour $x - 1$ is missing.** So:

- If $x - 1$ is in the set, skip x: someone ahead of it will measure this run.
- If $x - 1$ is not in the set, x starts a run. Walk right while $x + 1, x + 2, \ldots$ are present, and record the length.

Walk through `[100, 4, 200, 1, 3, 2]`. The set is {100, 4, 200, 1, 3, 2}.

x	Is $x-1$ present?	Action	Run length
100	99: no	walk: 101 missing	1
4	3: yes	skip	
200	199: no	walk: 201 missing	1
1	0: no	walk: 2, 3, 4 present, 5 missing	4
3	2: yes	skip	
2	1: yes	skip	

Best: **4**. (The real loop visits the set in scrambled order, but the result does not depend on the order.)

Why is this $O(n)$? Count the work. Each number is **checked as a possible start** once: n lookups. Each number is **walked over** at most once, because only the front of its run walks, and runs do not overlap. So at most n more lookups. Total about $2n$ lookups at $O(1)$ each: $O(n)$ **time** on average. Building the set is also $O(n)$. **Space:** the set holds up to n numbers, $O(n)$.

3.5 C++ solution

```cpp
// Daily Quant Digest 2026-10-29: Longest Consecutive Sequence (hash set, O(n) average)
// Compile: g++ -std=c++17 -O2 -Wall 2026-10-29.cpp && ./a.out
#include <algorithm>
#include <cassert>
#include <iostream>
#include <random>
#include <unordered_set>
#include <vector>

// Optimal: put every number in a hash set, then start counting only at the
// first number of each run (a number x whose left neighbour x-1 is missing).
// Values are within [-1e9, 1e9], so x - 1 and x + length never overflow an int.
int longestConsecutive(const std::vector<int>& nums) {
    std::unordered_set<int> present(nums.begin(), nums.end());  // duplicates vanish
    int best = 0;
    for (int x : present) {
        if (present.count(x - 1)) continue;   // not the start of a run: skip
        int length = 1;                       // x starts a run: walk right
        while (present.count(x + length)) ++length;
        best = std::max(best, length);
    }
    return best;
}

// Brute force for checking: sort, remove duplicates, then scan once.
int longestBySorting(std::vector<int> nums) {       // by value: we sort our own copy
    if (nums.empty()) return 0;
    std::sort(nums.begin(), nums.end());
    nums.erase(std::unique(nums.begin(), nums.end()), nums.end());
    int best = 1, run = 1;
    for (std::size_t i = 1; i < nums.size(); ++i) {
        if (nums[i] == nums[i - 1] + 1) ++run;
        else run = 1;
        best = std::max(best, run);
    }
    return best;
```

```cpp
}

int main() {
    // Examples from the problem statement
    struct Case { std::vector<int> nums; int expected; };
    std::vector<Case> cases = {
        {{100, 4, 200, 1, 3, 2}, 4},
        {{0, 3, 7, 2, 5, 8, 4, 6, 0, 1}, 9},
        {{}, 0},
        {{1, 2, 0, 1}, 3},
        {{-1000000000, 1000000000, 999999999, -999999999}, 2},
    };
    for (const auto& c : cases) {
        int got = longestConsecutive(c.nums);
        std::cout << "answer = " << got << " (expected " << c.expected << ")\n";
        assert(got == c.expected);
        assert(longestBySorting(c.nums) == c.expected);
    }

    // Randomized cross-check against the sorting version
    std::mt19937 rng(20261029);
    for (int trial = 0; trial < 20000; ++trial) {
        int n = rng() % 30;
        int range = 1 + rng() % 40;
        std::vector<int> nums(n);
        for (int& x : nums) x = static_cast<int>(rng() % range) - range / 2;
        assert(longestConsecutive(nums) == longestBySorting(nums));
    }

    // A large test: 1,000,000 numbers, a shuffled run of 0..999,999
    std::vector<int> big(1000000);
    for (int i = 0; i < 1000000; ++i) big[i] = i;
    std::shuffle(big.begin(), big.end(), rng);
    assert(longestConsecutive(big) == 1000000);

    std::cout << "All tests passed.\n";
    return 0;
}
```

Output:

```
answer = 4 (expected 4)
answer = 9 (expected 9)
answer = 0 (expected 0)
answer = 3 (expected 3)
answer = 2 (expected 2)
All tests passed.
```

The randomized test compares the hash-set answer with the sorting answer on 20,000 small random arrays full of duplicates and negatives. The last test builds a shuffled run of one million numbers and finishes in well under a second.

3.6 Walkthrough of the C++

`#include <unordered_set>`. The header for `std::unordered_set`. For the map version you would include `<unordered_map>`.

`int longestConsecutive(const std::vector<int>& nums)`. We take the array by **const reference** (Lesson 2): `&` means no copy is made, `const` promises we will not change it.

`std::unordered_set<int> present(nums.begin(), nums.end());`. This builds the set from a **range**: two iterators, the start and one past the end of the vector (Lesson 2). Every element is inserted, and duplicates are silently dropped, which is exactly what the problem wants. The longer way would be `std::unordered_set<int> present; for (int x : nums) present.insert(x);`. For a very large input you can also call `present.reserve(nums.size());` first, so the table does not have to grow (and rehash) several times.

`for (int x : present)`. A range-for works over any container, including a set. It visits each distinct key once, in no particular order. We copy each `int` into `x`, which is fine for small types. We loop over the **set**, not the original vector, so a number that appears 1,000 times is not tested 1,000 times.

`present.count(x - 1)`. For a set, `count` returns 1 if the key is there and 0 if not. In an `if`, 1 means true and 0 means false. In C++20 there is also `present.contains(x - 1)`, which reads better, but we stay with C++17. Another common style is `present.find(x - 1) != present.end()`: `find` returns an iterator to the key, or `end()` if it is missing.

`continue;`. Jumps straight to the next loop iteration. Here it means "not the front of a run, skip".

`while (present.count(x + length)) ++length;`. The walk to the right. We start with `length = 1` (just x) and check $x + 1$, $x + 2$, and so on. A `while` loop with a one-line body needs no braces, but be careful adding a second line later: it will not be inside the loop without braces.

Overflow note. The constraints say values are within $\pm 10^9$, and an `int` holds up to about 2.1×10^9, so `x - 1` and `x + length` are always safe. If the values could reach the `int` limits, `x - 1` on the smallest `int` would overflow, which is undefined behaviour in C++. Then you would do the arithmetic in `long long`.

`std::max(best, length)`. From `<algorithm>`: returns the larger of two values of the same type.

`int longestBySorting(std::vector<int> nums)`. This one takes the vector **by value** (no `&`): the function gets its own copy, so it may sort it without touching the caller's data.

`nums.erase(std::unique(nums.begin(), nums.end()), nums.end());`. The classic "remove duplicates" pair. On a sorted range, `std::unique` moves the distinct values to the front and returns an iterator to the new logical end, but does not shrink the vector. `erase(from, to)` then cuts off the leftover tail. Forgetting the `erase` is a well-known bug.

`std::size_t i`. The unsigned type used for sizes and indices. Comparing an `int` with `nums.size()` gives a compiler warning under `-Wall`, so we use the matching type.

`struct Case { std::vector<int> nums; int expected; };`. A tiny struct declared inside `main` just to hold test data. `{{100, 4, 200, 1, 3, 2}, 4}` fills it with **brace initialization**: the inner braces build the vector, the 4 is `expected`.

`for (const auto& c : cases).` `auto` lets the compiler work out the type (`Case`). `const auto&` avoids copying each test case and promises not to change it.

`assert(...).` From `<cassert>`: stops the program with a message if the condition is false. A cheap way to make tests loud.

`std::shuffle(big.begin(), big.end(), rng);.` Randomly reorders the vector using our seeded generator `rng` (Lesson 3), so the one-million test is not just a sorted array in disguise.

The `unordered_map` cousin. Many interview problems need a value with each key. For example, counting how many trades each trader did:

```
std::unordered_map<std::string, int> tradesBy;
for (const std::string& name : traderOfEachTrade) ++tradesBy[name];
```

`tradesBy[name]` creates the entry with value 0 the first time a name is seen, then `++` adds one. Handy, but note the side effect: even **reading** `tradesBy["Zed"]` creates an entry for Zed. To only look, use `count` or `find`.

---

4. Music: Jimi Hendrix, "All Along the Watchtower" (1968)

Context

"All Along the Watchtower" was written by **Bob Dylan**, who released it on his album *John Wesley Harding* at the end of 1967. Dylan's version is quiet and spare: acoustic guitar, harmonica, bass and drums. The Jimi Hendrix Experience recorded their version in early 1968 and released it on the double album *Electric Ladyland* (1968). It was his only single to reach the US Top 40, and it is the best-known recording of the song.

The story behind it

Hendrix admired Dylan deeply, and was given an early copy of *John Wesley Harding*. He heard something in this short, mysterious song and started recording it almost straight away, in London, at Olympic Studios. The session was loose: Dave Mason of the band Traffic played acoustic guitar, and Brian Jones of the Rolling Stones was in the room and added percussion. Bassist Noel Redding was not happy with the session and left, so Hendrix played the bass part himself. He then kept working on it for months in New York, adding more guitar parts as he went.

The best-known part of the story is what happened next. Dylan liked Hendrix's version so much that he started playing the song closer to the Hendrix arrangement in his own concerts, and he later said he had been doing it Hendrix's way ever since. It is rare for a songwriter of Dylan's stature to say that about a cover.

Why people love it

It is a masterclass in taking a great song and making it bigger without losing what made it great. Dylan wrote the words and the three-chord pattern. Hendrix kept both, and built a whole storm around them. The song is short, under four minutes, but it feels like an epic.

What to listen for

- **The opening.** Acoustic guitar strumming hard, then a crash and Hendrix's electric guitar slicing in. Tension from the first second.
- **The same few chords, all the way through.** The song circles around one short chord pattern. Notice how it never gets boring, because the guitar parts keep changing on top.
- **The solo in sections.** Hendrix's long solo moves through clearly different moods: a clean, singing melody, then a slide guitar passage, then a wah-wah section that sounds almost like a voice, then hard rhythmic chords. Try to hear where each one starts.
- **His singing.** Hendrix was shy about his voice, but his delivery here is calm and clear, which makes the noise around it more dramatic.
- **Compare.** Play Dylan's original straight after. Same song, two completely different worlds. A good lesson in how much arrangement matters.